

claude-windows / 7fce5410-4c57-4ec9-a054-be898bda22da

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\7fce5410-4c57-4ec9-a054-be898bda22da\subagents\workflows\wf\_c81dbe7a-7ff\agent-a66b4121dbf84bee5.jsonl`
- SHA-256: `418ac46a4fbd507da58f32298d9267c5ab30172041ebc96369b883b5a3e72a1f`
- Source modified: `2026-06-25T16:57:37+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_c81dbe7a-7ff`
- session_id: `7fce5410-4c57-4ec9-a054-be898bda22da`

Transcript

1. user (2026-06-25T16:56:32.124Z)

Adversarially verify this PR-review finding for PR #380 (branch o1-p11/gemini-remote-file-gc is checked out; read the real code with `git diff origin/master...HEAD` + Read the files). Be skeptical ? default to `isReal=false` / `adjustedSeverity=INVALID` if the finding is wrong, already handled by the code, or a non-issue. Do NOT re-run CI.

Finding [LOW] Unbounded `client.files.list()` on the startup path deviates from the repo's own "hang-killer" timeout convention
Location: `backend/providers/gemini.py:61`
Detail: `cleanup_orphaned_files()` calls `list(self.client.files.list())` with no bounded timeout, synchronously on the startup path (`main.py:1345`, before the CLI/server branch). The docstring and PR body both promise the sweep "never blocks startup," but the try/except only guarantees it never **raises** ? a slow or hanging network list call can still STALL startup until the SDK's default timeout. This contradicts the module's own established convention: the `analyze_video` Gemini calls are explicitly wrapped in `run_bounded/ExecutionPolicy` with a `HARD op_timeout` (lines 161-178, comment: "the hang-killer that was missing: a `generate()` that never returns previously blocked forever"). The new code is the one Gemini network call in this file NOT bounded. Impact is limited (best-effort, only when `ai_provider==gemini` and a key is set), so LOW, but it is a real observability/latency edge versus the repo's bar.
Suggested fix: Either bound the `list()` call with the existing `ExecutionPolicy/run_bounded` helper (already imported at line 15) so a network hang can't stall startup, or at minimum amend the docstring/comment to say "never raises" rather than "never blocks startup," so the claim matches behavior.

Return `isReal + adjustedSeverity` (INVALID if not real) + one-paragraph reasoning grounded in the actual code.

2. assistant (2026-06-25T16:56:34.606Z)

I'll verify this finding by examining the actual code on the PR branch.

3. user (2026-06-25T16:56:36.216Z)

```
diff --git a/backend/providers/gemini.py b/backend/providers/gemini.py
index 2ea0834..428bca8 100644
--- a/backend/providers/gemini.py
+++ b/backend/providers/gemini.py
@@ -2,6 +2,7 @@ import json
import logging
import os
import time
+from datetime import datetime, timedelta, timezone
+from typing import Tuple

from google import genai
```

```

@@ -40,6 +41,48 @@ class GeminiProvider(AIVideoProvider):
    except Exception:
        pass

+ def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
+     """Sweep and delete video files orphaned on the Gemini File API by a previous
+     crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
+     leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
+     that reach a ``finally``/except).
+
+     Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND whose
+     creation is older than ``grace_sec`` (so we never race a concurrent worker mid-upload).
+     Never raises; returns the count deleted. Intended to run once at startup.
+
+     Args:
+         grace_sec: only delete files older than this (default 1h). Protects a file a
+             live worker may still be processing.
+     """
+     cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
+     deleted = 0
+     try:
+         files = list(self.client.files.list())
+     except Exception as e:
+         logger.warning("[Gemini GC] failed to list remote files (skipping sweep): %s", e)
+         return 0
+     for f in files:
+         try:
+             mime = getattr(f, "mime_type", "") or ""
+             if not mime.startswith("video/"):
+                 continue
+             # f.create_time is an RFC3339 string on the File API.
+             ct = getattr(f, "create_time", None)
+             if ct is None:
+                 # No timestamp ? can't prove it's stale; leave it (safe default).
+                 continue
+             created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
+             if created > cutoff:
+                 continue # too recent ? a live worker may own it
+             self.client.files.delete(name=f.name)
+             deleted += 1
+         except Exception as e:
+             logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
+     if deleted:
+         logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
+     return deleted
+
+ def analyze_video(self, video_path: str, prompt: str,
+                 chunk_duration: float = 0.0,
+                 label: str = "") -> Tuple[list, dict]:

```

4. user (2026-06-25T16:56:36.922Z)

```

f00b71e feat(01/P11): Gemini remote-file GC ? startup orphan sweep
86705c1 chore: ruff format gemini_refine.py + trajectory_hybrid.py (#379)
b72ac85 fix(B2/P17): surface Gemini provider failures in all callers (#378)
71a8241 chore(repo): fix corrupted .gitignore (UTF-16 tail) + ignore artifacts/, track product
overview (#377)
f7fb36e docs(audit): fix two accuracy nits ? D2 DoD wording + main.py line count (#376)
---BRANCH---
01-p11/gemini-remote-file-gc

```

5. assistant (2026-06-25T16:56:39.365Z)

Now let me read the actual usage on the startup path to verify the claims about where this is

called and how the surrounding Gemini calls are bounded.

6. user (2026-06-25T16:56:40.996Z)

(Bash completed with no output)

7. user (2026-06-25T16:56:41.485Z)

```
1     import json
2     import logging
3     import os
4     import time
5     from datetime import datetime, timedelta, timezone
6     from typing import Tuple
7
8     from google import genai
9     from google.genai import types
10
11     from backend.infrastructure.execution_policy import (
12         ExecutionPolicy,
13         PolicyTimeout,
14         poll_until,
15         run_bounded,
16     )
17     from backend.providers.base import AIVideoProvider
18
19     logger = logging.getLogger(__name__)
20
21     # The Gemini File API caps uploads at 2GB; stay well under it. Callers must chunk so
22     # each uploaded clip is below this. Oversize uploads are refused (not silently
23     # attempted).
24     MAX_UPLOAD_MB = 500
25
26     class GeminiProvider(AIVideoProvider):
27         def __init__(self, api_key: str, model_name: str):
28             super().__init__(api_key, model_name)
29             # Initialize client lazily to avoid connection/auth issues during class
30             # instantiation
31             self._client = None
32
33         @property
34         def client(self):
35             if self._client is None:
36                 self._client = genai.Client(api_key=self.api_key)
37             return self._client
38
39         def _safe_delete(self, video_file) -> None:
40             """Best-effort cleanup of an uploaded File (never raises)."""
41             try:
42                 self.client.files.delete(name=video_file.name)
43             except Exception:
44                 pass
45
46         def cleanup_orphaned_files(self, grace_sec: int = 3600) -> int:
47             """Sweep and delete video files orphaned on the Gemini File API by a previous
48             crashed run (a hard process death between ``files.upload`` and ``_safe_delete``
49             leaves the uploaded clip behind forever ? the per-call cleanup only covers paths
50             that reach a ``finally``/except).
51
52             Best-effort: lists every file, deletes any whose ``mime_type`` is a video AND
53             whose
54             creation is older than ``grace_sec`` (so we never race a concurrent worker mid-
55             upload).
56
57             Never raises; returns the count deleted. Intended to run once at startup.
```

```

54         Args:
55             grace_sec: only delete files older than this (default 1h). Protects a file a
56                 live worker may still be processing.
57         """
58         cutoff = datetime.now(timezone.utc) - timedelta(seconds=grace_sec)
59         deleted = 0
60         try:
61             files = list(self.client.files.list())
62         except Exception as e:
63             logger.warning("[Gemini GC] failed to list remote files (skipping sweep):
%s", e)
64             return 0
65         for f in files:
66             try:
67                 mime = getattr(f, "mime_type", "") or ""
68                 if not mime.startswith("video/"):
69                     continue
70                 # f.create_time is an RFC3339 string on the File API.
71                 ct = getattr(f, "create_time", None)
72                 if ct is None:
73                     # No timestamp ? can't prove it's stale; leave it (safe default).
74                     continue
75                 created = datetime.fromisoformat(str(ct).replace("Z", "+00:00"))
76                 if created > cutoff:
77                     continue # too recent ? a live worker may own it
78                 self.client.files.delete(name=f.name)
79                 deleted += 1
80             except Exception as e:
81                 logger.debug("[Gemini GC] skipping a file during sweep: %s", e)
82         if deleted:
83             logger.info("[Gemini GC] deleted %d orphaned video file(s) older than %ds.",
deleted, grace_sec)
84         return deleted
85
86     def analyze_video(self, video_path: str, prompt: str,
87                       chunk_duration: float = 0.0,
88                       label: str = "") -> Tuple[list, dict]:
89         log_prefix = f"Gemini {label}" if label else "Gemini"
90
91         metrics: dict = {
92             "upload_time": 0.0,
93             "process_time": 0.0,
94             "gen_time": 0.0,
95         }
96
97         # 0. Refuse oversize uploads (Gemini File API limit is 2GB; we cap lower for
safety/speed)
98         try:
99             size_mb = os.path.getsize(video_path) / (1024 * 1024)
100         except OSError:
101             size_mb = 0.0
102         if size_mb > MAX_UPLOAD_MB:
103             logger.error(f"[{log_prefix}] {video_path} is {size_mb:.0f}MB >
{MAX_UPLOAD_MB}MB cap ? "
104                         f"SKIPPED WITHOUT ANALYSIS (no segments for this time range). "
105                         f"Re-encode/downscale chunks (indexing.ai_chunk_scale_width) or
chunk smaller.")
106             metrics["error"] = f"oversize: {size_mb:.0f}MB > {MAX_UPLOAD_MB}MB cap"
107             metrics["oversize_skipped"] = True
108             return [], metrics
109
110         # 1. Upload (retry transient network failures, e.g. WinError 10053 / reset
connections)
111         logger.info(f"[{log_prefix}] Uploading {video_path}...")
112         t_upload_start = time.time()

```

```

113         video_file = None
114         for attempt in range(3):
115             try:
116                 video_file = self.client.files.upload(file=video_path)
117                 metrics["upload_time"] = time.time() - t_upload_start
118                 logger.info(f"[{log_prefix}] Upload complete in
{metrics['upload_time']:.1f}s. URI: {video_file.uri}")
119                 break
120             except Exception as e:
121                 logger.warning(f"[{log_prefix}] Upload attempt {attempt + 1}/3 failed:
{e}")
122                 if attempt < 2:
123                     time.sleep(2 * (attempt + 1))
124         if video_file is None:
125             logger.error(f"[{log_prefix}] Upload failed after 3 attempts.")
126             metrics["error"] = "upload failed after 3 attempts"
127             return [], metrics
128
129         # 2. Wait for processing on Google servers ? a BOUNDED poll (a stuck file must
not hang
130         # the worker forever). Per-poll logs at DEBUG (no INFO spam); see
EXECUTION_POLICY.md.
131         t_process_start = time.time()
132         process_policy = ExecutionPolicy(poll_every_n_sec=10.0, max_wait_sec=1200.0) #
20 min cap
133
134         def _processed():
135             nonlocal video_file
136             video_file = self.client.files.get(name=video_file.name)
137             return None if video_file.state.name == "PROCESSING" else video_file
138
139         try:
140             video_file = poll_until(_processed, process_policy,
label=f"[{log_prefix}] processing", logger=logger)
141         except PolicyTimeout:
142             logger.error(f"[{log_prefix}] Processing exceeded
{process_policy.max_wait_sec:.0f}s ? giving up.")
143             self._safe_delete(video_file)
144             metrics["error"] = f"processing timed out after
{process_policy.max_wait_sec:.0f}s"
145             return [], metrics
146         except Exception as e:
147             logger.error(f"[{log_prefix}] Status check failed: {e}")
148             self._safe_delete(video_file)
149             metrics["error"] = f"status check failed: {e}"
150             return [], metrics
151         if video_file.state.name == "FAILED":
152             logger.error(f"[{log_prefix}] Video processing failed:
{video_file.error.message}")
153             self._safe_delete(video_file)
154             metrics["error"] = f"video processing failed: {video_file.error.message}"
155             return [], metrics
156
157         metrics["process_time"] = time.time() - t_process_start
158         logger.info(f"[{log_prefix}] Processing finished in
{metrics['process_time']:.1f}s.")
159
160         # 3. Call Gemini under a HARD per-attempt timeout (op_timeout_sec ? the hang-
killer that was
161         # missing: a generate() that never returns previously blocked forever) + retry
the 503/network
162         # FAILURE waves (capacity 503s on the flagship flash come in bursts; ~6
attempts).
163         logger.info(f"[{log_prefix}] Submitting multimodal request to
{self.model_name}...")

```

```

165         t_gen_start = time.time()
166         gen_policy = ExecutionPolicy(op_timeout_sec=180.0, max_retries=5,
backoff_base_sec=3.0)
167         raw_text = None
168
169         def _generate():
170             return self.client.models.generate_content(
171                 model=self.model_name,
172                 contents=[video_file, prompt],
173                 config=types.GenerateContentConfig(
174                     response_mime_type="application/json", temperature=0.0),
175             )
176
177         try:
178             response = run_bounded(_generate, gen_policy, label=f"{log_prefix}
generate", logger=logger)
179             raw_text = response.text
180             metrics["gen_time"] = time.time() - t_gen_start
181             logger.info(f"[{log_prefix}] Response received in
{metrics['gen_time']:.1f}s:\n{raw_text}")
182         except Exception as e:                                # PolicyTimeout (hang) or the last 503 ?
both end here
183             logger.error(f"[{log_prefix}] generate gave up after retries: {e}")
184             metrics["error"] = f"generate failed after retries: {e}"
185             metrics["generate_failed"] = True
186             self._safe_delete(video_file)
187
188         if not raw_text:
189             return [], metrics
190
191         # 4. Parse JSON
192         try:
193             segments = json.loads(raw_text)
194             if not isinstance(segments, list):
195                 logger.error(f"[{log_prefix}] Response is not a JSON array.")
196                 metrics["error"] = "response is not a JSON array"
197                 return [], metrics
198             return segments, metrics
199         except Exception as e:
200             logger.error(f"[{log_prefix}] JSON parse failed: {e}")

```

8. assistant (2026-06-25T16:56:45.415Z)

The main.py diff returned empty, which means `cleanup\_orphaned\_files` is NOT called from main.py in this PR. Let me verify where it's actually invoked.

9. user (2026-06-25T16:56:46.481Z)

```

backend\providers\gemini.py:44:     def cleanup_orphaned_files(self, grace_sec: int = 3600) ->
int:
backend\main.py:1200:         deleted = provider.cleanup_orphaned_files()
tests\test_provider_gemini.py:119: def test_cleanup_orphaned_files_deletes_stale_videos_only():
tests\test_provider_gemini.py:134:         deleted =
provider.cleanup_orphaned_files(grace_sec=3600)
tests\test_provider_gemini.py:139: def test_cleanup_orphaned_files_list_failure_is_swallowed():
tests\test_provider_gemini.py:145:         assert provider.cleanup_orphaned_files() == 0
tests\test_provider_gemini.py:148: def
test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
tests\test_provider_gemini.py:162:         deleted =
provider.cleanup_orphaned_files(grace_sec=3600)
tests\test_provider_gemini.py:202:     provider.cleanup_orphaned_files.side_effect =
RuntimeError("api down")
tests\test_provider_gemini.py:207:     provider.cleanup_orphaned_files.assert_called_once()

```

10. user (2026-06-25T16:56:46.926Z)

```
backend/main.py | 25 ++++++++
backend/providers/gemini.py | 43 ++++++
tests/test_provider_gemini.py | 104 ++++++
3 files changed, 172 insertions(+)
```

11. assistant (2026-06-25T16:56:49.776Z)

The call site is `backend/main.py:1200`, not `main.py:1345` as the finding states. The finding's line number is wrong but the file path direction is right (backend/main.py). Let me read the actual call site and surrounding context to understand the startup path and whether it blocks.

12. user (2026-06-25T16:56:50.584Z)

```
1175         any work ? a fatal incoherent profile (e.g. cloud-serving pointed at local-only
1176         storage) OR, for
1177         the exposed server (`include_exposure=True`, M9 hardening), a half-configured
1178         exposure (edge
1179         auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when
1180         no
1181         ``deployment.profile`` is selected AND edge auth is off (today's behaviour). No GPU
1182         probe here ?
1183         the server stays torch-light; the detector healthcheck is the GPU authority.
1184         Extracted so the
1185         fail-fast wiring + the default-OFF no-op are unit-testable.""
1186         try:
1187             enforce_startup_checks(config, logger=logger, include_exposure=True)
1188         except StartupCheckError as e:
1189             logger.error("[STARTUP] refusing to start: %s", e)
1190             sys.exit(1)
1191
1192         def _maybe_sweep_gemini_orphans(config) -> None:
1193             """O1/P11: best-effort sweep of Gemini File-API uploads orphaned by a previous
1194             crashed run.
1195             A no-op unless the configured provider is Gemini AND its API key is present. Never
1196             raises ?
1197             a failed sweep is logged and swallowed so it can't block startup. Extracted for unit
1198             tests.""
1199             try:
1200                 provider_name = getattr(config, "ai_provider", "")
1201                 if provider_name != "gemini":
1202                     return
1203                 api_key = os.environ.get("GEMINI_API_KEY")
1204                 if not api_key:
1205                     return
1206                 provider = provider_registry.get("gemini", api_key=api_key,
1207                 model_name=config.ai_model)
1208                 deleted = provider.cleanup_orphaned_files()
1209                 if deleted:
1210                     logger.info("[STARTUP] Gemini orphan sweep reclaimed %d remote file(s).",
1211                     deleted)
1212             except Exception as e:
1213                 logger.warning("[STARTUP] Gemini orphan sweep skipped (non-fatal): %s", e)
1214
1215         def main():
1216             parser = argparse.ArgumentParser(description="Sports Highlight Indexer
1217             Orchestrator")
1218             parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
1219             parser.add_argument("--output-dir", default="output", help="Directory where
1220             processed clips/highlights are saved")
1221             parser.add_argument("--setup", action="store_true", help="Launch dependency checker
1222             and setup utility")
```

```

1212     parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
1213     parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
1214     parser.add_argument("--app", action="store_true",
1215                          help="Launch the app: start the local server (localhost-only)
and open the "
1216                               "bundled UI in your browser. Falls back to a free port if
--port is busy.")
1217     parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
1218     parser.add_argument("--host", default=None,
1219                          help="Bind address. Default 127.0.0.1 (localhost-only); 0.0.0.0
when "
1220                               "security.edge_auth_enabled. Expose on a network ONLY
behind an auth gate.")
1221     parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
1222
1223     available_segmenters = list(segmenter_registry.list_available().keys())
1224     parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
1225     parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
1226     parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
1227     parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
1228
1229     args = parser.parse_args()
1230
1231     if args.setup:
1232         # Invoke the diagnostics/bootstrap script (renamed from the old root
1233         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
1234         # build system). Resolve its path from this file so `indexer --setup`

```

13. assistant (2026-06-25T16:56:53.645Z)

Now let me see exactly where `\_maybe\_sweep\_gemini\_orphans` is called within `main()` to determine if it's truly on the startup path before the CLI/server branch.

14. user (2026-06-25T16:56:54.595Z)

```

backend\main.py:1188:def _maybe_sweep_gemini_orphans(config) -> None:
backend\main.py:1345:     _maybe_sweep_gemini_orphans(global_config)
tests\test_provider_gemini.py:167:# --- 01/P11: main.py startup wiring
(_maybe_sweep_gemini_orphans) ---
tests\test_provider_gemini.py:178:     main_mod._maybe_sweep_gemini_orphans(cfg)
tests\test_provider_gemini.py:191:     main_mod._maybe_sweep_gemini_orphans(cfg)
tests\test_provider_gemini.py:206:     main_mod._maybe_sweep_gemini_orphans(cfg)

```

15. assistant (2026-06-25T16:56:59.599Z)

The finding's line `main.py:1345` refers to the **call** to `\_maybe\_sweep\_gemini\_orphans` within `main()`, not the **list()** call itself (which is at gemini.py:61). The finding correctly cited gemini.py:61 for the unbounded list and main.py:1345 for the startup invocation. Let me read the call site context at 1345 to verify it's before the CLI/server branch.

16. user (2026-06-25T16:57:01.076Z)

```

1330         f"start ? a second instance would corrupt in-flight job state. Stop
the other one, or "
1331         f"use a different --output-dir (RALLY_APP_HOME) to run an independent
instance.")

```



```

1332         sys.exit(1)
1333
1334     db_instance = Database(db_path)
1335
1336     # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
1337     # the executor is in-process. Mark them failed so pollers see a terminal state.
1338     swept = db_instance.fail_stale_jobs()
1339     if swept:
1340         logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
1341
1342     # 01/P11: sweep Gemini File-API uploads orphaned by a hard process death (the per-
call
1343     # _safe_delete covers normal/except paths, but a SIGKILL/OOM mid-upload leaves a
clip on
1344     # Google's servers forever). Best-effort, video-only, grace-gated; never blocks
startup.
1345     _maybe_sweep_gemini_orphans(global_config)
1346
1347     # CLI processing mode (no web UI needed). --app/--server always run the server,
never CLI single-shot.
1348     if args.video_path and not args.server and not args.app:
1349         print(f"[CLI] Processing video file: {args.video_path}")
1350         # M2: bare/local path = identity (today's behaviour); a bucket/Drive URI is
fetched to scratch.
1351         # The local-existence fast-fail only applies to a local input (stat the RESOLVED
key, not the
1352         # raw URI); a remote fetch fails loudly. An unsupported scheme is a clean error,
not a crash.
1353         storage_cfg = getattr(global_config, "storage", None)
1354         try:
1355             input_ref = storage.resolve_input_ref(args.video_path, storage_cfg)
1356         except ValueError as e:
1357             print(f"[FAIL] {e}")
1358             return
1359         if input_ref.backend == "local" and not os.path.exists(input_ref.key):
1360             print(f"[FAIL] Video file not found: {args.video_path}")
1361             return
1362         local_video = storage.acquire_local_input(args.video_path, storage_cfg)
1363
1364         video_id = compute_video_id(local_video)
1365         was_reingest = db_instance.get_video(video_id) is not None
1366
1367         # Validate (with-context variant surfaces ffprobe_meta for the report)
1368         print("[CLI] Validating...")
1369         val_results, val_context = registry.run_all_with_context(local_video,
global_config)
1370         failures = [r for r in val_results if not r.passed]
1371
1372         # Save video status
1373         status = "failed" if failures else "processed"
1374         db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])

```

17. assistant (2026-06-25T16:57:04.988Z)

I now have the full picture. The finding's line numbers are accurate: `gemini.py:61` is the unbounded `list()`, `main.py:1345` is the startup invocation before the CLI/server branch (line 1348). The comment at line 1344 literally says "never blocks startup."

Let me verify the `run\_bounded`/`ExecutionPolicy` import claim and check whether the SDK `client.files.list()` call could actually hang (does the genai client have a default timeout). Let me also confirm the imports the finding references.

18. user (2026-06-25T16:57:06.333Z)

```

1      """Per-job execution policy: bounded, observable waits for long/flaky ops
(EXECUTION_POLICY.md P1).
2
3      Turns an unbounded blocking call (the GX010125 Gemini hang) into a bounded one:
4      - ``run_bounded`` ? a HARD per-attempt timeout (``op_timeout_sec``, the hang-killer) +
retry-with-backoff
5          for FAILURES (an exception) *and* HANGS (no return in time). A hung attempt's worker
thread is a daemon,
6          so it is abandoned, not joined ? it never blocks the process.
7      - ``poll_until`` ? a bounded poll loop for an in-progress external op
(``poll_every_n_sec`` cadence,
8          ``max_wait_sec`` deadline).
9
10     **Testability:** ``sleep``/``monotonic`` are injected, so the whole module is unit-
tested with zero real
11     waiting (a fake clock advances instantly).
12
13     **Logging discipline** (so polling is VISIBLE but never NOISY): every poll/retry logs at
**DEBUG**; state
14     transitions (start, resolved, hung, gave-up) log at **INFO/WARNING**. INFO stays clean;
enable DEBUG on the
15     ``execution.policy`` logger to watch the poll trail.
16     """
17     from __future__ import annotations
18
19     import logging
20     import threading
21     import time as _time
22     from dataclasses import asdict, dataclass
23     from enum import Enum
24     from typing import Callable, Optional, TypeVar
25
26     T = TypeVar("T")
27
28     #: Dedicated logger so callers can dial polling visibility independently of the app's
root level.
29     LOG = logging.getLogger("execution.policy")
30
31
32     class WaitMode(str, Enum):
33         """How a job treats a long wait. ``str``-valued so it serialises straight to JSON on
the job row."""
34         WAIT_AND_RESUME = "wait_and_resume" # default: bounded wait; over budget ->
reschedule/partial (P2/P3)
35         ABORT = "abort" # one attempt; on hang/fail, stop immediately
36         BLOCK = "block" # bounded in-process blocking wait (no
reschedule)
37
38
39     class PolicyTimeout(TimeoutError):
40         """An op hung past ``op_timeout_sec`` or never became ready within
``max_wait_sec``."""
41
42
43     @dataclass(frozen=True)
44     class ExecutionPolicy:
45         """Declarative resilience knobs, attachable per job/op (JSON-serialisable; see
EXECUTION_POLICY.md)."""
46         mode: WaitMode = WaitMode.WAIT_AND_RESUME
47         poll_every_n_sec: float = 10.0
48         op_timeout_sec: float = 120.0 # HARD per-attempt deadline ? the hang-killer
that was missing
49         max_wait_sec: float = 1800.0 # total wall budget for a poll loop
50         max_retries: int = 5 # retries (=> max_retries+1 attempts) for
failures/hangs

```

```

51         backoff_base_sec: float = 3.0          # exponential: backoff_base * 2**attempt
52         on_timeout: str = "reschedule"         # reschedule | partial | fail (consumed by
P2/P3)
53         resumable: bool = True
54
55         def to_dict(self) -> dict:
56             d = asdict(self)
57             d["mode"] = self.mode.value
58             return d
59
60         @classmethod
61         def from_dict(cls, d: dict) -> "ExecutionPolicy":
62             known = {f for f in cls.__dataclass_fields__} # type: ignore[attr-defined]
63             kw = {k: v for k, v in d.items() if k in known}
64             if "mode" in kw and not isinstance(kw["mode"], WaitMode):
65                 kw["mode"] = WaitMode(kw["mode"])
66             return cls(**kw)
67
68
69         #: The sane default (also what a job gets if it declares no policy).
70         DEFAULT_POLICY = ExecutionPolicy()

```

19. assistant (2026-06-25T16:57:11.107Z)

This confirms the codebase's "hang-killer" convention is real and is exactly what the finding describes: ``run_bounded`` provides a HARD per-attempt timeout via a daemon thread that gets abandoned (not joined) if it hangs, so a hung network call never blocks the process. The module docstring explicitly references "the GX010125 Gemini hang" as the motivating incident.

Now let me confirm the test coverage ? does any test exercise a `*hanging* `list()`` (vs just a raising one)? This tells me whether the "never blocks" claim is actually tested.

20. user (2026-06-25T16:57:12.134Z)

```

115         f.state.name = state
116         return f
117
118
119     def test_cleanup_orphaned_files_deletes_stale_videos_only():
120         """Deletes video files older than the grace window; leaves non-videos and fresh
files."""
121         from datetime import datetime, timedelta, timezone
122         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
123         fresh = (datetime.now(timezone.utc) - timedelta(minutes=5)).isoformat()
124         files = [
125             _file_mock("old-vid", "video/mp4", stale),          # stale video -> deleted
126             _file_mock("fresh-vid", "video/quicktime", fresh), # fresh video -> kept (grace)
127             _file_mock("old-img", "image/png", stale),          # non-video -> kept
128             _file_mock("notts", "video/mp4", None),             # no timestamp -> kept (safe)
129         ]
130         with patch("backend.providers.gemini.genai"):
131             provider = GeminiProvider(api_key="k", model_name="m")
132             provider._client = MagicMock()
133             provider._client.files.list.return_value = iter(files)
134             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
135             assert deleted == 1
136             provider._client.files.delete.assert_called_once_with(name="old-vid")
137
138
139     def test_cleanup_orphaned_files_list_failure_is_swallowed():
140         """A list() failure must not raise ? the sweep is best-effort and must not block
startup."""
141         with patch("backend.providers.gemini.genai"):
142             provider = GeminiProvider(api_key="k", model_name="m")
143             provider._client = MagicMock()

```

```

144         provider._client.files.list.side_effect = RuntimeError("network")
145         assert provider.cleanup_orphaned_files() == 0
146
147
148     def test_cleanup_orphaned_files_per_file_delete_failure_is_swallowed():
149         """A delete() failure on one file must not abort the sweep."""
150         from datetime import datetime, timedelta, timezone
151         stale = (datetime.now(timezone.utc) - timedelta(hours=2)).isoformat()
152         files = [
153             _file_mock("a", "video/mp4", stale),
154             _file_mock("b", "video/mp4", stale),
155         ]
156         with patch("backend.providers.gemini.genai"):
157             provider = GeminiProvider(api_key="k", model_name="m")
158             provider._client = MagicMock()
159             provider._client.files.list.return_value = iter(files)
160             # First delete raises, second succeeds ? _safe_delete swallows it.
161             provider._client.files.delete.side_effect = [RuntimeError("boom"), None]
162             deleted = provider.cleanup_orphaned_files(grace_sec=3600)
163             assert deleted == 1 # only the second file counted
164
165
166
167     # --- 01/P11: main.py startup wiring (_maybe_sweep_gemini_orphans) ---
168
169     def test_maybe_sweep_skips_non_gemini_provider(monkeypatch):
170         """A non-Gemini provider must short-circuit (no client built, no sweep)."""
171         import backend.main as main_mod
172         cfg = MagicMock()
173         cfg.ai_provider = "openai"
174         called = []
175         monkeypatch.setattr(main_mod.provider_registry, "get",
176                             lambda *a, **k: called.append(k) or MagicMock())
177         monkeypatch.setenv("GEMINI_API_KEY", "k")
178         main_mod._maybe_sweep_gemini_orphans(cfg)
179         assert called == [] # provider never instantiated
180
181
182     def test_maybe_sweep_skips_when_no_api_key(monkeypatch):
183         """No GEMINI_API_KEY -> no sweep (can't auth)."""
184         import backend.main as main_mod
185         cfg = MagicMock()
186         cfg.ai_provider = "gemini"
187         monkeypatch.delenv("GEMINI_API_KEY", raising=False)
188         called = []
189         monkeypatch.setattr(main_mod.provider_registry, "get",
190                             lambda *a, **k: called.append(k) or MagicMock())
191         main_mod._maybe_sweep_gemini_orphans(cfg)
192         assert called == []
193
194
195     def test_maybe_sweep_runs_and_swallows_errors(monkeypatch):
196         """Gemini + key present -> sweep runs; a provider error is swallowed (never blocks
197 startup)."""
198         import backend.main as main_mod
199         cfg = MagicMock()
200         cfg.ai_provider = "gemini"
201         cfg.ai_model = "gemini-2.5-flash"
202         provider = MagicMock()
203         provider.cleanup_orphaned_files.side_effect = RuntimeError("api down")
204         monkeypatch.setattr(main_mod.provider_registry, "get", lambda *a, **k: provider)
205         monkeypatch.setenv("GEMINI_API_KEY", "k")
206         # Must not raise.
207         main_mod._maybe_sweep_gemini_orphans(cfg)
208         provider.cleanup_orphaned_files.assert_called_once()

```

21. assistant (2026-06-25T16:57:24.382Z)

The tests confirm the analysis: every test exercises a **raising** failure (``side_effect = RuntimeError``), never a **hanging** call. There is no test that a slow/hanging ``list()`` is bounded, because nothing bounds it. The try/except guarantees "never raises" but not "never blocks."

I have everything I need to make a verdict. Let me synthesize:

****The finding is accurate on the technical substance:****

1. ``gemini.py:61`` ? ``list(self.client.files.list())`` has no bounded timeout. Confirmed.
2. It runs synchronously on the startup path: ``main()`` ? line 1345 ``_maybe_sweep_gemini_orphans`` ? line 1200 ``cleanup_orphaned_files``, which is ****before**** the CLI/server branch at line 1348. Confirmed.
3. The try/except (lines 60-64) only guarantees it never **raises** ? a network hang in ``list()`` would stall before reaching the except. Confirmed.
4. The comment at `main.py:1344` explicitly claims "never blocks startup," and the docstring at `gemini.py:52` says "Intended to run once at startup" with "Never raises." The `main.py` comment's "never blocks startup" is the overclaim. Confirmed.
5. The repo's own convention (``run_bounded`/`ExecutionPolicy`` with ``op_timeout_sec``, the "hang-killer") is real, imported at `gemini.py:11-16`, used for the ``analyze_video`` generate call (lines 161-178), and this new ``list()`` is indeed the one Gemini network call in the file NOT bounded. Confirmed.

****Severity calibration:**** The impact is genuinely small. The genai SDK does set HTTP timeouts by default (httpx-based, with connect/read timeouts), so a true infinite hang is unlikely ? but the convention exists precisely because the team was burned by a Gemini call that hung forever (GX010125, referenced in `execution_policy.py`). It only fires when ``ai_provider == gemini`` and a key is set, and only at startup. LOW is appropriate.

The finding's two minor inaccuracies (it says "main.py:1345" which is actually ``backend/main.py``, and the prose loosely conflates that with the `list()` line) do not undermine it ? both line numbers it cites are correct for what they describe.

22. user (2026-06-25T16:57:32.894Z)

Structured output provided successfully

23. assistant (2026-06-25T16:57:36.947Z)

Verdict submitted: the finding is ****real**** at ****LOW**** severity. The unbounded ``list(self.client.files.list())`` at ``backend/providers/gemini.py:61`` does run synchronously on the startup path (``backend/main.py:1345`` ? ``:1200``, before the CLI/server branch at ``:1348``), the try/except only guarantees "never raises" not "never blocks startup" (as the ``main.py:1344`` comment claims), and this is the lone Gemini network call in the file not wrapped in the repo's established ``run_bounded`/`ExecutionPolicy`` hang-killer convention. Impact is genuinely small (gemini-only, key-gated, best-effort, SDK has default HTTP timeouts), so LOW stands.